

Hardware:

- *3D Printed Robot Base Specs*
- *Base size*: 12 × 19 cm, custom 3D printed
- *Motor mounts*: 2 mounting holes on each side for N20 encoder motors
- *Wheel cutouts*: 72 mm cutouts made for 65 mm diameter wheels
- *Wheel adapters*: 3D-printed adapters connect motors to wheels due to D-shaped shaft vs rectangular wheel hub mismatch
- *Battery compartment*: Located on underside to keep battery low and improve stability
- *Caster wheel mounts*: 2 mounting holes at the rear
- *IR sensor slots*: 8 rectangular slots at the front to secure IR reflective sensors with zip ties
- *Color sensor mounts*: 2 additional mounting holes on each side in front of IR slots, for green-turn detection
- *Wiring access*: Large rectangular opening behind sensor area for routing all sensor wiring to central ESP32 controller.



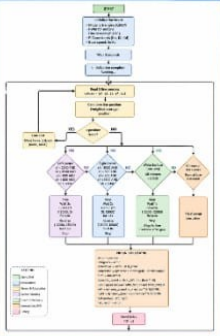
Challenges

One of our biggest problem was when the orders we put in didn't arrive quickly. This led to lots of delay on the robot and we couldn't start on anything as all the other parts like video and poster were dependant on the robot.



Software:

- *Line-Following & Control Logic*
- *Sensor setup*: 5-sensor array reads line position
- *Position calc*: Uses weighted average of sensor readings to find robot's position relative to line
- *PID control*: Position used as error input
 - *P term*: Corrects current deviation from line
 - *D term*: Improves stability by reacting to rate of change
 - *I term*: Kept minimal to avoid drift
- *Motor adjustment*: PID adjusts left + right motor speeds to keep robot centered
- *Gap handling*: If all sensors see white → treats as gap, aligns to last black line, then moves forward until line is found again
- *Sharp corners*: Detected by specific sensor patterns → triggers controlled turn + brief reverse to reposition over the turn
- *Color sensor logic*:
 - Green on one side → robot turns that direction
 - Green on both sides → robot makes U-turn
- *Speed control*: Robot slows down during turns to avoid overshooting the line
- *Result*: Combined methods let it navigate lines, corners, and gaps smoothly in maze
- *Next step*: Add camera to detect ball in evacuation zone + place it in correct box



Our team:

- Zain: I handle most of the coding and part of the mechanical build, focusing on efficient code and safety features so the robot stays on track.
- Naksh: I led the mechanical design and building, testing LEGO Spike → M5 → 3D printed ESP32 designs while learning robot programming.
- Abigail: I managed the poster and team updates, breaking down scoring and qualification requirements using my RoboCup Junior build/programming experience.
- Jwalitha: I designed the poster, and researched competition format and interview prep while helping build key robot parts.



Time line

Beginning	Planing	Set up	Building	Finish
We came 3rd place in Linköping robocup using lego spike robot (block programming) and qualified for the World Championships.	We started ordering more equipment and tools such as sensors and servos from other countries	We gathered all the necessary equipment and structured the robot. We made a clear sketch of the robot	We 3D printed a base. Mounted holes for each component and started creating a stable base and added all the wires and sensor.	We finished week 26-27! We continued adding more final touches as well as attaching the claw

ESP32

ESP32 is better than SPIKE Prime for line rescue because its higher refresh rate improves PID line-following accuracy and makes the robot react faster. It is also more customizable, giving full freedom to . ESP32 also is easier to debug, which speeds up testing and improvement. The custom chassis is more stable than spike heavy hub. Overall, these factors contribute to why Esp32 is a better source and why we choose it. ESP32 is better than SPIKE Prime for line rescue because its higher refresh rate improves PID line-following accuracy and makes the robot react faster. It is also more customizable , giving full freedom to . ESP32 also is easier to debug, which speeds up testing and improvement. The custom chassis is more stable than spike heavy hub. Overall, these factors contribute to why Esp32 is a better source and why we choose it

SPIKE robot:

Spike robot, Linköping. We used this robot in the Swedish National Robocup!

The robot has a simple programming and structure making it easy to adjust and it doesn't overcomplicate the different codes. The size of the robot is very small and the components are not heavy or long making it easy to pass through multiple obstacles

The first problem was a lack of ports which meant that we couldn't add the arm to it. If we were supposed to fix this problem we needed to combine the two hubs together, and that needed python which at that time none of the team members were confident in using python. The second challenge was the ball caster which was made of plastic which didn't give enough force to the front wheels so they were able to cross the ramp. The third challenge was the hub, which having a large mass, led to a disbalance in the weight which meant more force from motors and was the reason why the robot in Linköping was not able to complete the ramps.

